

week5

1. Status 업데이트 전략

1.1. 왜 Status가 중요한가

Spec은 원하는 상태, Status는 관찰된 현재 상태

Status 을 보는 주체

- 사용자 — `kubectl get memo` 출력, `kubectl describe`
- 다른 컨트롤러 — Argo Rollouts가 Deployment status를 보고 progress 판단하듯
- 자기 자신 — 다음 Reconcile에서 "이전에 어디까지 했더라" 참조

예시

```
# Deployment의 status - Argo Rollouts/HPA/kubectl이 이걸 봄
status:
  replicas: 3
  readyReplicas: 3
  updatedReplicas: 3
  availableReplicas: 3
  conditions:
  - type: Progressing
    status: "True"
    reason: NewReplicaSetAvailable
```

1.2. Status Subresource

```
// +kubebuilder:subresource:status
type Memo struct { ... }
```

이게 있으면:

- `r.Status().Update()` 로 status만 업데이트 가능 (spec 안 건드릴림)
- 사용자가 `kubectl edit memo` 로 status 수정해도 무시됨 (컨트롤러만 쓰는 영역으로 분리)
- RBAC을 `memos/status` 서브리소스 단위로 분리 가능

없으면 `r.Update()` 로 spec/status가 같이 갱신되어 race condition이 생기기 쉽습니다.

1.3. Status 필드 설계 패턴

나쁜 예 — 의미 없는 단일 문자열:

```
type MemoStatus struct {
    Phase string `json:"phase"` // "Running"? "OK"? 뭘 의미하는지 모름
```

```
}
```

좋은 예 — 구조화된 관찰값:

```
type MemoStatus struct {
    // 1. 관찰된 핵심 수치
    ReadyReplicas int32 `json:"readyReplicas"`

    // 2. 자식 리소스 참조
    ConfigMapName string `json:"configMapName,omitempty"`

    // 3. 마지막으로 처리한 spec 버전 (역등성 디버깅용)
    ObservedGeneration int64 `json:"observedGeneration,omitempty"`

    // 4. 표준화된 Conditions (가장 중요)
    Conditions []metav1.Condition `json:"conditions,omitempty" patchStrategy:"merge" patchMergeKey:"type"`
}
```

1.4. ObservedGeneration 패턴

가장 중요한 status 필드 중 하나입니다.

- `metadata.generation` — Spec이 바뀔 때마다 쿠버네티스가 증가시키는 카운터
- `status.observedGeneration` — 컨트롤러가 "내가 이 generation까지 처리했음"이라고 기록하는 값

```
// Reconcile 끝에서
memo.Status.ObservedGeneration = memo.Generation
r.Status().Update(ctx, &memo)
```

사용처:

```
# 사용자가 spec 바꿨는데 status가 반영됐는지 확인
kubectl get memo my-memo -o jsonpath='{.status.observedGeneration}=={.metadata.generation}'
```

Argo Rollouts가 `progressDeadlineSeconds` 판단할 때 이 패턴을 사용합니다. "내가 본 spec generation"과 "현재 spec generation"이 다르면 아직 처리 중이라고 판단.

1.5. Conditions — 쿠버네티스 표준 패턴

Status의 표준 표현 방식

```
type Condition struct {
    Type          string          // 무엇에 대한 상태인가
    Status        ConditionStatus // "True" | "False" | "Unknown"
    Reason        string          // 머신 친화적 이유 (CamelCase)
```

```

    Message      string           // 사람 친화적 설명
    LastTransitionTime metav1.Time // 상태가 바뀐 시각
    ObservedGeneration int64      // 어느 generation에서 관찰됐는지
}

```

Type 예시	True 의미
Ready	사용 가능한 상태
Available	가용성 충족
Progressing	작업 진행 중
ConfigMapSynced	ConfigMap이 동기화됨
ExternalAPIReachable	외부 API 접근 가능

설정 헬퍼 (apimachinery 제공):

```

import "k8s.io/apimachinery/pkg/api/meta"

meta.SetStatusCondition(&memo.Status.Conditions, metav1.Condition{
    Type:      "Ready",
    Status:    metav1.ConditionTrue,
    Reason:    "ConfigMapCreated",
    Message:   fmt.Sprintf("ConfigMap %s is ready", cm.Name),
    ObservedGeneration: memo.Generation,
})

```

SetStatusCondition :

- 같은 Type이 이미 있으면 업데이트
- Status가 *바뀐 경우에만* LastTransitionTime 갱신 (이게 핵심 — 불필요한 status update 방지)
- 없으면 추가

조회 헬퍼:

```

cond := meta.FindStatusCondition(memo.Status.Conditions, "Ready")
if cond != nil && cond.Status == metav1.ConditionTrue { ... }

```

1.6. Status Update vs Patch

Status 업데이트할 때도 race condition이 있습니다.

```

// 문제: Reconcile 도중 다른 무언가가 CR을 수정함
r.Get(ctx, ..., &memo)           // resourceVersion = 100
// ... 작업 ...
r.Status().Update(ctx, &memo)     // 그 사이 다른 컨트롤러가 105로 올림 → Conflict

```

해결책 두 가지:

(1) Conflict 발생 시 재시도 (단순)

```
return ctrl.Result{}, err // controller-runtime이 알아서 재시도
```

(2) Patch 사용 (안전)

```
patch := client.MergeFrom(memo.DeepCopy())
memo.Status.ReadyReplicas = 3
r.Status().Patch(ctx, &memo, patch)
```

Patch는 resourceVersion 비교 없이 "이 필드만 바꿔달라"고 요청해서 conflict가 거의 안 납니다. **프로덕션에선 Patch 권장.**

1.7. Status 업데이트 빈도 제어

Reconcile마다 무조건 Status를 Update하면:

- API 서버 부하
- 다른 컨트롤러가 또 Reconcile 트리거됨 (status도 watch 이벤트를 발생시키므로)
- etcd 쓰기 폭증

원칙: Status가 실제로 바뀐 경우에만 업데이트.

```
original := memo.DeepCopy()

// ... status 필드들 수정 ...

if !reflect.DeepEqual(original.Status, memo.Status) {
    if err := r.Status().Patch(ctx, &memo, client.MergeFrom(original)); err != nil {
        return ctrl.Result{}, err
    }
}
```

1.8. Printer Columns

kubectl get 에서 status를 노출하려면:

```
// +kubebuilder:printcolumn:name="Ready",type=string,JSONPath=`.status.conditions[?(@.type=="Ready")].status`
// +kubebuilder:printcolumn:name="Replicas",type=integer,JSONPath=`.status.readyReplicas`
// +kubebuilder:printcolumn:name="Age",type=date,JSONPath=`.metadata.creationTimestamp`
type Memo struct { ... }
```

결과:

```
$ kubectl get memo
NAME          READY  REPLICAS  AGE
my-memo       True   3          5m
```

ESO `ExternalSecret`, Argo `Rollout` 이 다 이렇게 노출합니다.

2. Finalizer 패턴

▼ OwnerReference

OwnerReference는 자식 리소스의 `metadata.ownerReferences` 에 부모 객체를 표현하는 표식

```
metadata:
  name: my-configmap
  ownerReferences:
  - apiVersion: blog.abouthere.kr/v1alpha1
    kind: Memo
    name: my-memo
    uid: abc-123
    controller: true
    blockOwnerDeletion: true
```

2.1. 왜 OwnerReference로 부족한가

- AWS Secrets Manager에 만들어둔 시크릿 삭제
- S3 버킷 삭제
- 외부 DB의 row 삭제
- 외부 모니터링 시스템에 등록 해제
- 사내 시스템 (Tidy, Secrethub) 호출

2.2. Finalizer가 뭐가

`metadata.finalizers` 는 문자열 배열입니다. 이 배열이 비어있지 않으면 쿠버네티스는 객체를 실제로 삭제하지 않습니다.

```
metadata:
  name: my-memo
  finalizers:
  - memo.blog.abouthere.kr/cleanup-external
  deletionTimestamp: "2026-06-04T10:00:00Z" # 삭제 요청은 됐지만...
```

흐름:

1. 사용자 `kubectl delete memo my-memo` 실행
2. 쿠버네티스: `deletionTimestamp` 설정 + 그 외엔 안 건드릴

3. Finalizer가 있으니 실제 삭제는 보류

- 컨트롤러: deletionTimestamp 감지 → 외부 정리 작업 수행
- 컨트롤러: 정리 완료 후 finalizer 문자열 제거
- finalizers가 빈 배열이 되면 → 쿠버네티스가 실제로 etcd에서 삭제

2.3. 표준 구현 패턴

```
const memoFinalizer = "memo.blog.abouthere.kr/cleanup-external"

func (r *MemoReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    var memo blogv1alpha1.Memo
    if err := r.Get(ctx, req.NamespacedName, &memo); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }

    // 1. 삭제 진행 중인가?
    if !memo.DeletionTimestamp.IsZero() {
        // 1-1. 우리 finalizer가 붙어있다면 정리 수행
        if controllerutil.ContainsFinalizer(&memo, memoFinalizer) {
            if err := r.cleanupExternalResources(ctx, &memo); err != nil {
                return ctrl.Result{}, err // 실패 시 재시도
            }
            // 1-2. 정리 완료 → finalizer 제거
            controllerutil.RemoveFinalizer(&memo, memoFinalizer)
            if err := r.Update(ctx, &memo); err != nil {
                return ctrl.Result{}, err
            }
        }
        // 정상 종료 (이후 쿠버네티스가 실제 삭제)
        return ctrl.Result{}, nil
    }

    // 2. 생성/수정 흐름 - finalizer가 없으면 추가
    if !controllerutil.ContainsFinalizer(&memo, memoFinalizer) {
        controllerutil.AddFinalizer(&memo, memoFinalizer)
        if err := r.Update(ctx, &memo); err != nil {
            return ctrl.Result{}, err
        }
        // 다음 Reconcile에서 본 로직으로
        return ctrl.Result{}, nil
    }

    // 3. 일반 reconcile 로직 (4주차에서 만든 것)
```

```
// ...

return ctrl.Result{}, nil
}

func (r *MemoReconciler) cleanupExternalResources(ctx context.Context, memo *
blogv1alpha1.Memo) error {
    // 외부 API 호출
    // 멍등하게 짤 것 - 이미 삭제됐어도 에러 안 나와 함
    return nil
}
```

2.4. Finalizer 이름 규칙

쿠버네티스 규칙: `<domain>/<name>` 형식. 도메인이 있는 형태로 짓는 게 표준입니다.

- ✓ memo.blog.abouthere.kr/cleanup-external
- ✓ secrethub.abouthere.kr/release-secret
- ✗ cleanup
- ✗ my-finalizer

도메인이 없으면 `kubernetes.io` 같은 예약된 네임스페이스로 오인될 수 있어 webhook이 거부할 수도 있어요.

2.5. cleanupExternalResources의 황금률

반드시 멍등(idempotent)해야 합니다. Reconcile이 재시도되면 cleanup도 여러 번 호출되니까요.

```
func (r *MemoReconciler) cleanupExternalResources(ctx context.Context, memo *
blogv1alpha1.Memo) error {
    err := externalAPI.Delete(memo.Status.ExternalID)
    if err != nil && !isNotFoundError(err) {
        return err // 진짜 에러만 반환
    }
    return nil // 이미 없으면 OK로 처리
}
```

`isNotFoundError` 같은 에러는 성공으로 간주. 그래야 두 번째 호출 때도 깔끔하게 끝납니다.

2.6. 흔한 함정

함정 1: Finalizer 영구 잔존 (Stuck)

컨트롤러에 버그가 있어서 finalizer를 제거 못 하면, 사용자가 `kubectl delete` 를 해도 객체가 영영 안 사라집니다.

`Terminating` 상태로 멈춰있죠.

```
$ kubectl get memo
NAME          STATUS
```

```
my-memo    Terminating    # 5시간째 이려고 있음
```

응급조치 (학습 환경에서만):

```
kubectl patch memo my-memo -p '{"metadata":{"finalizers":null}}' --type=merge
```

이러면 정리 작업 건너뛰고 강제 삭제됩니다. 프로덕션에선 외부 리소스가 orphan으로 남으니까 신중하게.

함정 2: Finalizer 추가 후 즉시 cleanup 로직 실행

```
// ❌ 잘못된 순서
if !memo.DeletionTimestamp.IsZero() {
    cleanup(...)
}
if !controllerutil.ContainsFinalizer(&memo, ...) {
    AddFinalizer(&memo, ...)
    r.Update(ctx, &memo)
}
```

삭제 시 finalizer가 없을 수도 있는 케이스를 못 잡습니다. **항상 deletionTimestamp 체크가 먼저**, finalizer 추가는 그 뒤.

함정 3: Finalizer 추가와 자식 생성을 한 Reconcile에서

```
// ❌ 위험
AddFinalizer(&memo, ...)
r.Update(ctx, &memo)           // Update 1 (spec/metadata)
createExternalResource(...)    // 외부 API
r.Status().Update(ctx, &memo)  // Update 2 (status)
```

Finalizer Update와 외부 리소스 생성 사이에 컨트롤러가 죽으면 finalizer 없이 외부 리소스만 남습니다. **항상 finalizer 먼저 Update하고 return** → 다음 Reconcile에서 실제 작업.

```
// ✅ 안전
if !ContainsFinalizer(&memo, ...) {
    AddFinalizer(&memo, ...)
    if err := r.Update(ctx, &memo); err != nil {
        return ctrl.Result{}, err
    }
    return ctrl.Result{}, nil // 명시적으로 끊고 다음 reconcile 트리거
}
// 이 시점엔 finalizer가 확실히 붙어있음
createExternalResource(...)
```

함정 4: 외부 정리 실패 시 무한 재시도

외부 시스템이 영구적으로 응답 안 하면 finalizer가 영영 안 빠집니다. 보통 다음으로 대응:

- 일정 횟수 재시도 후 Condition에 `CleanupFailed` 기록 → 사용자가 대시보드에서 보고 수동 개입
- alert 발송
- `RequeueAfter` 로 backoff 늘리기

3. Garbage Collection (GC) 동작 원리

3.1. 세 가지 삭제 정책 (Propagation Policy)

`kubectl delete` 시 `--cascade` 옵션으로 선택:

정책	동작
Background (기본값)	부모를 즉시 삭제 마킹, 자식은 GC가 백그라운드에서 비동기 삭제
Foreground	자식을 먼저 모두 삭제, 다 끝나면 부모 삭제
Orphan	부모만 삭제, 자식은 그대로 둠 (OwnerReference 끊김)

```
kubectl delete memo my-memo --cascade=background # 기본
kubectl delete memo my-memo --cascade=foreground # 자식까지 다 사라질 때까지 명령이 안 끝남
kubectl delete memo my-memo --cascade=orphan # 자식 남김
```

3.2. Foreground GC와 blockOwnerDeletion

`SetControllerReference` 가 자동으로 붙여주는 `blockOwnerDeletion: true` 가 Foreground 정책에서 동작합니다.

"이 자식이 살아있는 동안 부모를 못 지운다"

내부 동작:

1. 사용자가 Memo 삭제 (foreground)
2. Memo에 `foregroundDeletion` finalizer 자동 추가 (쿠버네티스가)
3. ConfigMap의 `blockOwnerDeletion: true` 가 부모 삭제를 막음
4. GC가 ConfigMap을 삭제
5. ConfigMap이 사라지면 부모를 막던 요소가 없어짐
6. Memo의 `foregroundDeletion` finalizer 제거
7. Memo 실제 삭제

3.3. Finalizer vs OwnerReference 선택 기준

상황	사용
자식이 쿠버네티스 리소스 (ConfigMap, Secret, Deployment 등)	OwnerReference
자식이 외부 시스템 (AWS, DB, 사내 API)	Finalizer
둘 다 있음	둘 다 사용 (병행 가능)

ESO의 ExternalSecret 예시:

- 만들어내는 쿠버네티스 Secret → OwnerReference
- AWS Secrets Manager 쪽엔 보통 손 안 댄다 → Finalizer 불필요

만약 ExternalSecret 삭제 시 AWS 쪽 시크릿도 지우게 만들고 싶다면 → Finalizer 추가.